

Online Exam Simulator

With Anti-Cheat System

Course Title: Software Engineering Sessional

Submitted By

Team Name: Team 'Double Quotation'
Department: Computer Science & Engineering
Institute: North Bengal International University,
Rajshahi.

Team Members:	ID:
Md. Razab Ali	0712340012101011
Md. Shakibul Hasan	0712340012101028
Mst. Sanjida Emros	0712340012101037
Md. Polash Ali	0712340012101045
Md. Raja Hossein	0712340012101046

Submitted To:

Course Instructor: Rifa Zumana
Designation: Lecturer
Department: Computer Science & Engineering
Institute: North Bengal International University,
Rajshahi.

"Dedicated to building secure, reliable, and high-performance academic assessment solutions."

Submission Date: 18 MAY 2026

Online Exam Simulator with Anti-Cheat System

Team “Double Quotation”

1. Introduction

With the increasing demand for online education, ensuring the integrity of examinations has become a significant challenge. Traditional browser-based online exam systems are often susceptible to various cheating methods, including tab switching, screen recording, and the use of unauthorized external applications. This project addresses these vulnerabilities by proposing and implementing a robust desktop-based online examination software. The system is designed to enforce a controlled exam environment while maintaining seamless online connectivity for centralized data management, prioritizing security, reliability, and practical applicability in real-world academic settings.

2. Motivation / Requirement

The primary motivation for this project stems from the inherent weaknesses in existing online examination systems. These systems frequently suffer from:

- **Limited Control:** Over-reliance on web browsers provides insufficient control over the examination environment, making it easy for students to bypass security measures.
- **High Cheating Risk:** The ability to access system-level functions or external applications significantly increases the risk of academic dishonesty.
- **Ineffective Monitoring:** Current solutions often lack comprehensive mechanisms for monitoring student behavior during exams.
- **Internet Instability:** Fluctuations in internet connectivity can disrupt exams and compromise their integrity.

To counter these issues, there was a clear requirement for a system that could provide a secure, stable, and monitored examination platform, ensuring fair assessment and upholding academic standards.

3. Key Features

The Online Desktop Exam Simulator with Anti-Cheat System offers a comprehensive suite of features designed to enhance security, streamline exam management, and provide an intuitive user experience:

- **Role-Based Authentication:** Secure login mechanisms differentiate between student and teacher roles, providing appropriate access levels.
- **Full-Screen Enforced Exam Mode:** Students are required to take exams in a locked, full-screen environment, minimizing distractions and preventing access to unauthorized applications.
- **Advanced Anti-Cheat Detection:** The system incorporates sophisticated mechanisms to detect and log various forms of cheating, including:
 - Real-time monitoring of window switching and focus loss.
 - Detection of unauthorized keyboard inputs and shortcuts (e.g., Alt+Tab, Esc, Win key).
 - OS-level security checks, optimized for Windows environments.
 - Multiple login detection to prevent simultaneous access from different devices.
- **Dynamic Exam Management:** Teachers can create, store, and manage an unlimited number of questions, design flexible exam formats, and set custom timings.
- **Automated Evaluation and Reporting:** The system automatically grades multiple-choice questions and generates detailed reports, including integrity logs for detected violations.
- **Professional UI/UX:** Built with Tkinter, the application features a modern gradient design, responsive interface, intuitive workflows, and accessible visuals.
- **Secure Data Management:** Utilizes SQLite for secure local storage with encrypted credentials, password hashing, comprehensive test logs, and JSON configuration for system parameters.

4. Workflow and Data Flow Diagram

This project followed the **Incremental Development Model**. Since the system had multiple independent layers core login, exam flow, anti-cheat, security hardening, and UI polish each layer was built, tested, and validated before the next was started. This model was chosen because the anti-cheat system and security upgrades could only be meaningfully designed after the core exam functionality was stable, and because bugs caught in one iteration could not silently propagate into the next. It also allowed the working application to be demonstrated at any point during development, which is ideal for a project where features were continuously added and improved over time.

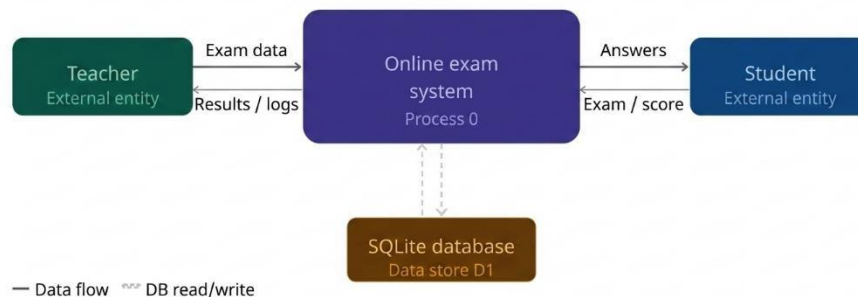
- 1 **User Role Selection:** Upon launching the application, users select either 'Student Portal' or 'Teacher Portal'.
- 2 **Authentication:** Users log in with their credentials. Teachers manage exams, while students access available exams.
- 3 **Teacher Workflow:**
 - **Exam Creation:** Teachers define exam details (name, subject, duration, total marks) and add questions. Questions are stored in the database.
 - **Exam Management:** Teachers can view, edit, and monitor their created exams.
 - **Violation Monitoring:** Teachers can access anti-cheat logs to review any detected student violations.

4 Student Workflow:

- **Exam Access:** Students view a list of available exams.
- **Exam Attempt:** Upon starting an exam, the system enters a full-screen locked mode, and the anti-cheat system is activated. Questions are presented, and student responses are recorded.
- **Submission:** Exams are automatically submitted upon timeout or manual submission. The anti-cheat system logs any violations during the exam.
- **Results Viewing:** Students can view their scores and performance after exam submission.

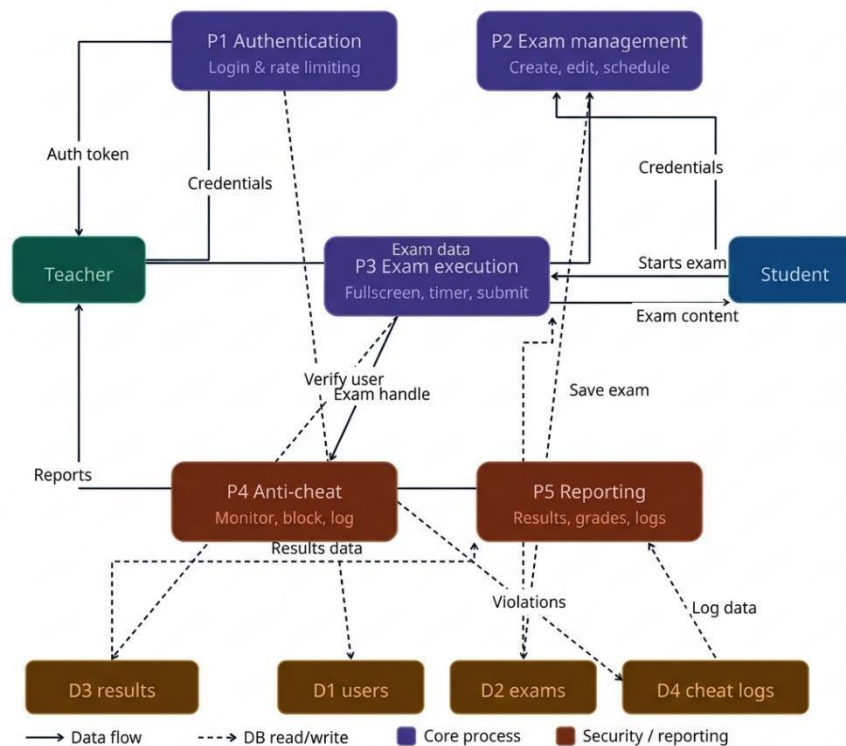
Data Flow Diagram (DFD) - Level 0: Context Diagram

Level 0 --- Context diagram

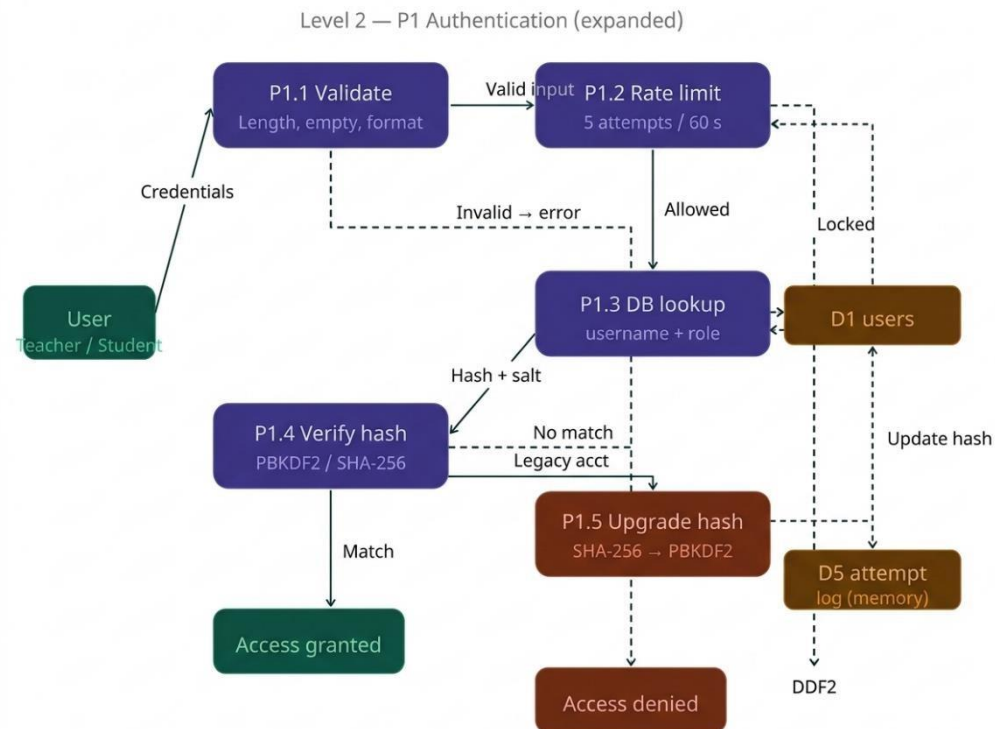


Data Flow Diagram (DFD) - Level 1: System Processes

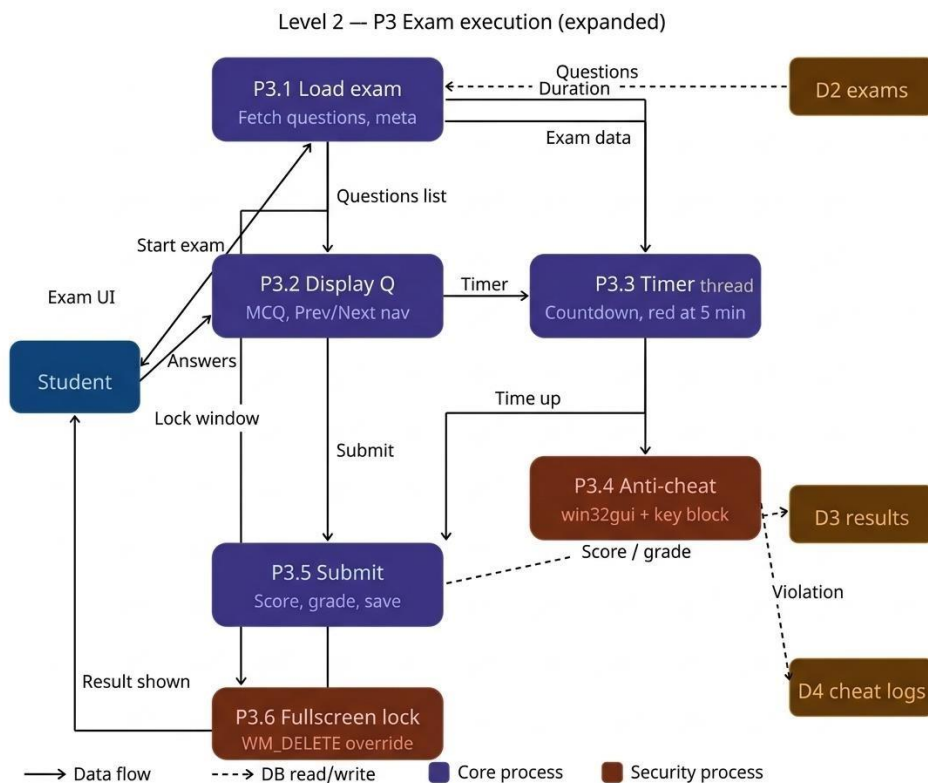
Level 1 --- System processes



Data Flow Diagram (DFD) - Level 2-P1:



Data Flow Diagram (DFD) - Level 2-P1:



5. Used Technologies and the GitHub link

The project leverages a robust set of technologies to deliver its features:

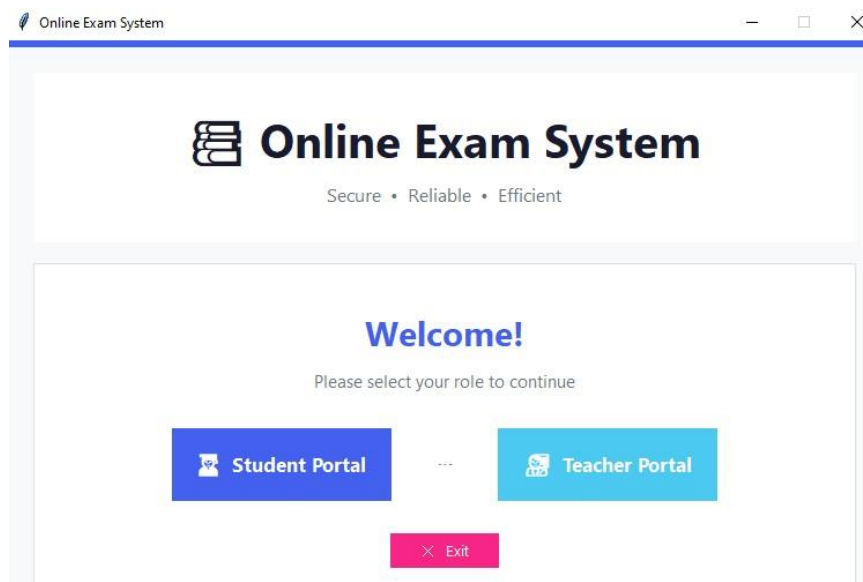
Component	Technology
Frontend	Tkinter (Python)
Database	SQLite3
Security	Hashlib, Keyboard Monitoring, Win32 API
Threading	Python Threading Module
Authentication	Password Hashing & Session Management

GitHub Repository: <https://github.com/RZBv2/Team-Double-Quotation>

6. Screenshots of the Key Features Working

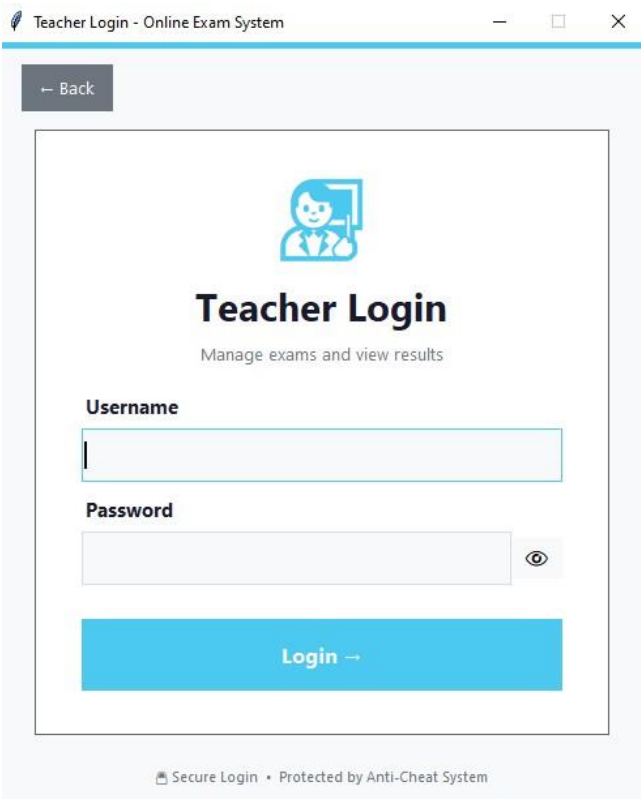
This section provides visual demonstrations of the system's key functionalities, presented sequentially to illustrate the user journey.

6.1. Role Selection Interface



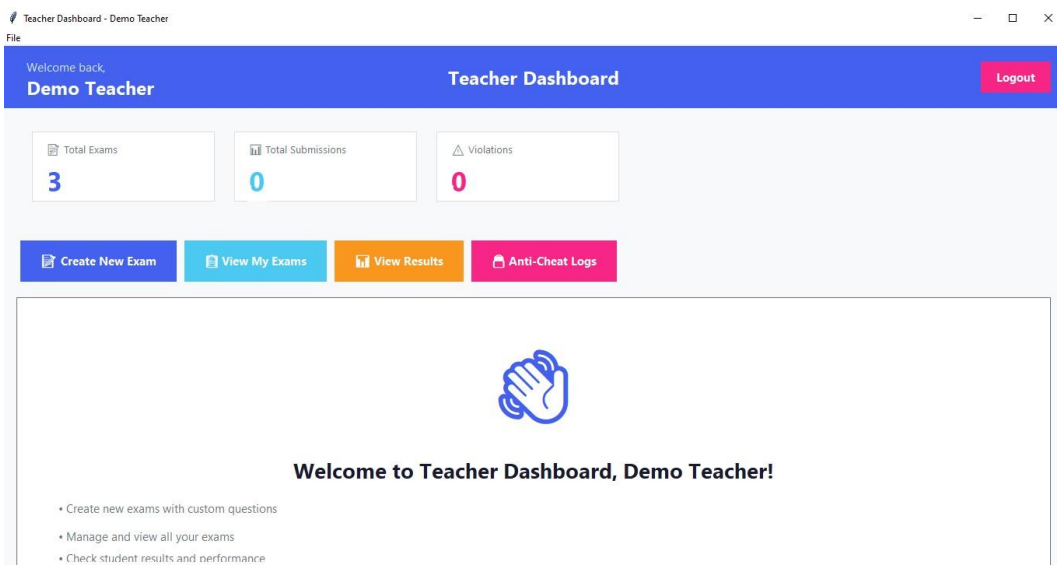
This screenshot displays the initial interface where users choose between the 'Student Portal' and 'Teacher Portal'. This role-based entry point ensures appropriate access control from the outset.

6.2. Teacher Login:



This image shows the dedicated login screen for teachers, featuring fields for username and password, along with a visual toggle for password visibility. The footer highlights the secure login and anti-cheat protection.

6.3. Teacher Dashboard



Upon successful login, teachers are presented with this dashboard. It provides an overview of total exams, submissions, and violations, along with navigation options to create new exams, view existing ones, check results, and review anti-cheat logs.

6.4. Teacher - My Exams

 **My Exams**
You have 3 exam(s) created

ID	Exam Name	Subject	Duration	Total Marks	Date	Status
1	Python Programming Basics	Programming	30	25	2026-05-16 21:52	pending
2	Mathematics Fundamentals	Mathematics	45	25	2026-05-17 21:52	pending
3	General Knowledge Quiz	General Knowledge	20	25	2026-05-18 21:52	pending

This screenshot illustrates the 'My Exams' section for teachers, displaying a tabular view of all created exams with details such as ID, name, subject, duration, total marks, date, and current status. This allows for easy management and monitoring of examinations.

6.5. Teacher - Create New Exam

 **Create New Exam**
Fill in the details to create a new examination

Exam Information

Exam Name:

Subject:

Questions

+ Add Question

Here, teachers can define the parameters for a new examination. The interface is divided into 'Exam Information' (for basic details like name and subject) and 'Questions' (where individual questions can be added).

6.6. Teacher - Anti-Cheat Logs

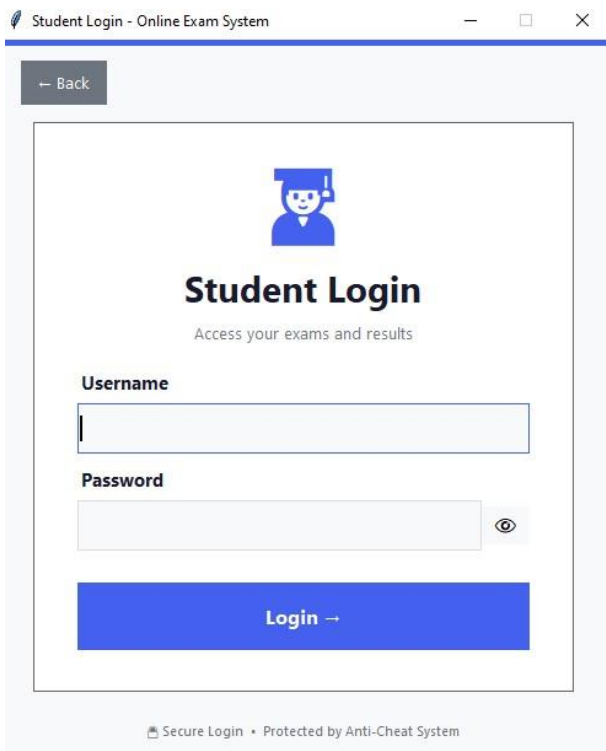


No anti-cheat logs available.

No violations detected yet

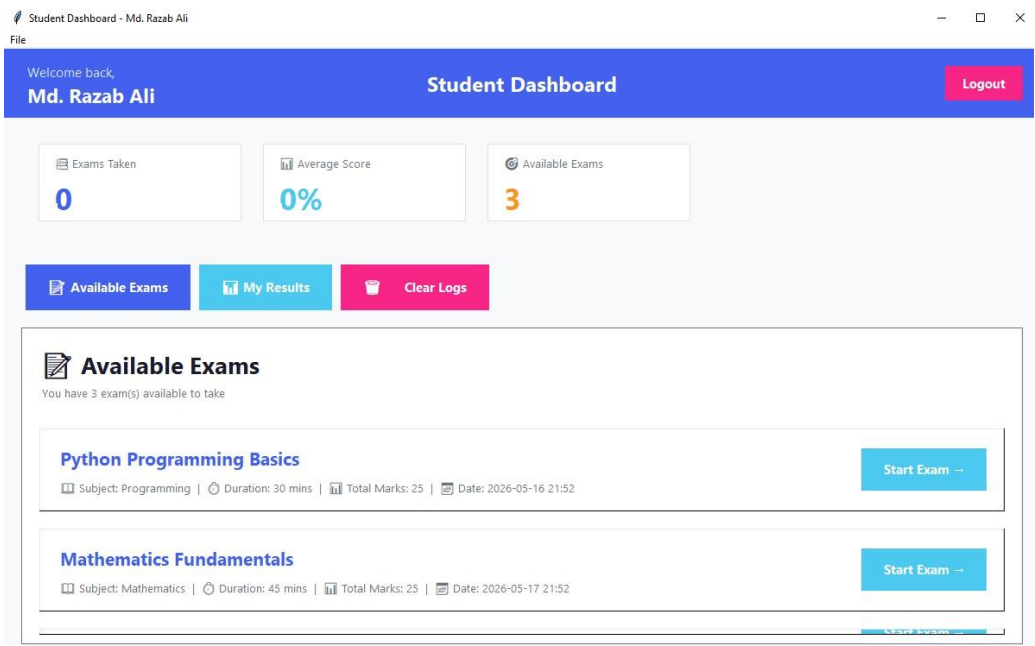
This view shows the anti-cheat log interface. In this specific instance, it indicates that no violations have been detected yet, demonstrating the system's capability to record and display such events when they occur.

6.7. Student Login:



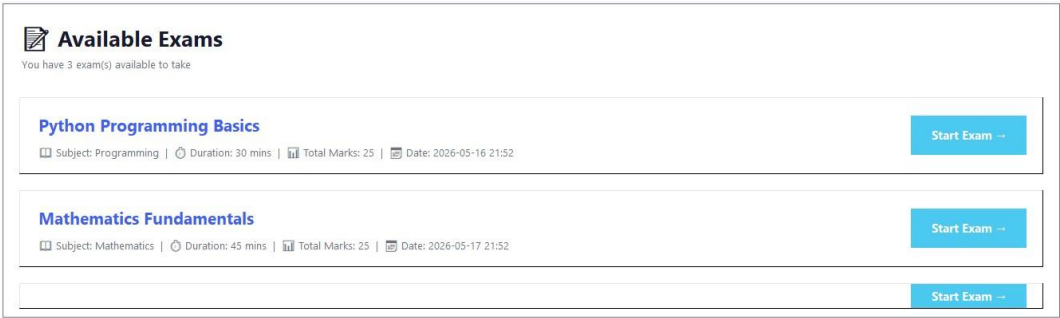
Similar to the teacher login, this is the secure authentication screen for students to access their portal, exams, and results.

6.8. Student Dashboard



After logging in, students land on their dashboard, which summarizes their exam activity (exams taken, average score) and lists available exams. Navigation options include 'Available Exams', 'My Results', and 'Clear Logs'.

6.9. Student - Available Exams



This screenshot presents the list of exams currently available for the student to take. Each entry provides key information like subject, duration, total marks, and date, with a prominent 'Start Exam' button.

6.10. Student - My Results

The screenshot shows a section titled 'My Results' with a sub-header 'You have taken 2 exam(s)'. Below this is a table with the following data:

Exam Name	Score	Total Marks	Percentage	Submitted At
Python Programming Basics	15	25	60.0%	2026-05-15 16:15:55

Finally, this image shows the 'My Results' section for students, detailing their performance in completed exams. It includes the exam name, score, total marks, percentage, and submission timestamp.

7. SQA and Testing Strategies

The development of this project incorporated several Software Quality Assurance (SQA) and testing strategies to ensure its reliability, security, and functionality. Given the iterative nature of development, where features were added and updated over time, an **Agile development model** with continuous integration and testing principles was implicitly followed. This allowed for early detection and resolution of issues.

Key testing strategies included:

- **Unit Testing:** Individual components and functions, such as password hashing, database interactions, and anti-cheat detection logic, were tested in isolation to ensure they performed as expected.
- **Integration Testing:** Different modules of the system (e.g., login with database, exam display with anti-cheat) were tested together to verify seamless interaction.
- **System Testing:** The entire application was tested as a complete system to ensure it met all specified requirements and behaved correctly in various scenarios.
- **Security Testing:** A critical focus was placed on security testing, particularly for the anti-cheat system. This involved attempting to bypass the anti-cheat mechanisms (e.g., using Alt+Tab, trying to switch windows, launching unauthorized applications) to identify vulnerabilities. For instance, an initial problem where the main anti-cheat system was not functioning correctly, allowing full-screen lock bypass and window switching, was identified and subsequently fixed through rigorous testing and debugging.
- **User Acceptance Testing (UAT):** Although not explicitly detailed, the iterative development and feedback process from the user (developer) on new features and fixes implies a form of UAT to ensure the software met the end-user's expectations and requirements.

8. Contributions, Limitations, and Future Work

Contributions

This project makes significant contributions to secure online examination systems by:

- **Developing a robust desktop-based solution:** Moving beyond browser limitations to offer a more controlled exam environment.
- **Implementing advanced OS-level anti-cheat mechanisms:** Directly addressing common cheating techniques like window switching and unauthorized key presses.
- **Providing a comprehensive management system:** Enabling teachers to efficiently create, manage, and monitor exams, along with detailed logging of student activities.
- **Focusing on UI/UX:** Delivering an intuitive and professional user interface for both students and teachers, enhancing usability.

Limitations

Despite its strengths, the current system has certain limitations:

- **Platform Dependency:** The advanced anti-cheat features are primarily optimized for Windows operating systems, limiting full functionality on other platforms.
- **Scalability:** While using SQLite for local storage is convenient for single-user or smallscale deployments, it may not be ideal for large-scale institutional use requiring concurrent access and robust server-side management.
- **Real-time Proctoring:** While window switching is monitored, advanced real-time proctoring features like facial recognition or AI-based behavior analysis are not yet fully integrated.

Future Work

Potential future enhancements for the system include:

- **Cross-Platform Anti-Cheat:** Expanding OS-level anti-cheat functionalities to macOS and Linux environments.
- **Advanced Proctoring Integration:** Incorporating facial recognition and AI-based behavior analysis for more sophisticated real-time proctoring.
- **Cloud-Based Backend:** Migrating to a more scalable backend infrastructure (e.g., FastAPI/Flask with MySQL) to support larger user bases and centralized data management.
- **Question Bank Enhancements:** Implementing more diverse question types and advanced question randomization algorithms.
- **Reporting and Analytics:** Developing more in-depth analytical tools for teachers to gain insights into student performance and exam integrity trends.

9. Discussion

Building this project was a genuine learning experience. Since features kept getting added and updated after the initial build, we naturally fell into an incremental workflow build something, test it, fix it, repeat. That flexibility turned out to be essential.

The biggest headache was the anti-cheat system. It simply didn't work at first full-screen mode wasn't locking properly, and students could switch windows without any consequence. That completely defeated the point. After some dedicated debugging, we tracked down the issues and got it working the way it was supposed to: fullscreen enforced, window switching detected, violations logged.

There was also a smaller but annoying UI problem button sizes were inconsistent across different screens, which made the app look unpolished. Not a critical bug, but the kind of thing you notice and it bothers you, so we fixed it.

Looking back, the anti-cheat struggle was probably the most valuable part of the whole project. It taught us that security features need to be tested hard, not just written and assumed to work.